QoS-Aware Multi-Fidelity Runtime for TinyML Inference under Dynamic Workload Contention

Narendra Satish

Abstract—Deploying deep neural networks on resource-constrained microcontrollers (TinyML) is increasingly challenged by dynamic CPU contention, sporadic burst workloads, and strict inference deadlines. Static deployment paradigms force engineers into an inflexible compromise: either deploying an overly conservative lightweight model that perpetually sacrifices accuracy, or deploying a high-capacity model that suffers deadline violations under transient CPU contention. In this paper, we develop and empirically evaluate a trace-driven, ground-truth-independent QoS-Aware Multi-Fidelity Runtime that dynamically selects among pre-verified Pareto-optimal TinyML models based on execution deadlines, workload contention, and user-defined Quality of Service (QoS) policies. Operating strictly without ground-truth label access during execution, our runtime manages three operational modes: FAST (student_a_8_4_fp32, 160 MACs), BALANCED (pruned_mlp_14f_75pct, 96 MACs), and HIGH_FIDELITY (student_b_16_4_fp32, 304 MACs). We formalize and evaluate four dynamic scheduling policies across an 80-configuration experimental sweep (5 deadlines $\times$ 4 workload contention regimes $\times$ 4 policies) evaluated over 11,200 held-out test frames. Our trace-driven simulation demonstrates that under high and burst contention, the BALANCED policy dynamically transitions from the dense model to the sparse model, achieving up to a 68.4% reduction in theoretical active arithmetic operations (MACs) (96 vs. 304 MACs per inference frame relative to the high-fidelity baseline) while matching or exceeding the diagnostic macro F1 score (0.7563 vs. 0.7390) with only a -0.37% accuracy difference. Furthermore, controlled ablation studies show that our QoS-aware runtime achieves a $+3.21\%$ accuracy improvement over static minimal execution and prevents starvation during modeled CPU bursts, demonstrating dynamic multi-fidelity selection as an effective systems approach for adaptive edge intelligence within the evaluated simulation regime.

Index Terms—TinyML, Quality of Service, Dynamic Model Selection, Multi-Fidelity Runtime, Embedded Systems, Microcontroller Deep Learning, Adaptive Edge AI.

I. Introduction

EMBEDDED Machine Learning and Tiny Machine Learning (TinyML) have expanded rapidly, enabling intelligent inference on resource-constrained microcontrollers (MCUs) at the extreme edge [1], [2]. In time-critical cyber-physical systems such as automotive powertrain control, industrial condition monitoring, and robotic sensing, embedded microcontrollers must execute neural network inference while simultaneously servicing high-priority interrupts, sensor acquisition timers, and communication protocols [3], [4].

Manuscript received August 20, 2026. (Corresponding author: Narendra Satish, e-mail: narendresh.p@gmail.com). All model binaries, runtime implementations, and simulation harnesses are fully reproducible.

Under conventional deployment paradigms, a single neural network architecture is statically compiled into microcontroller Flash storage [5], [6]. However, this static approach creates an acute systems-level dilemma under dynamic workload conditions:

1) Under-Provisioning Dilemma: Deploying a lightweight model ensures execution deadline compliance under worst-case CPU contention, but perpetually sacrifices classification accuracy when ample compute headroom is available [7], [8].
2) Over-Provisioning Dilemma: Deploying a high-capacity model maximizes diagnostic fidelity during nominal operation, but causes severe deadline misses and buffer overflows when sporadic bursts or interrupts saturate the microcontroller CPU [9], [10].

To address this trade-off, we develop a QoS-Aware Multi-Fidelity Runtime Architecture for embedded TinyML. Rather than relying on a static model, our runtime system maintains a pre-verified registry of Pareto-optimal models spanning multiple operational modes (FAST, BALANCED, and HIGH_FIDELITY). A lightweight, deadline-aware QoS scheduler dynamically assesses operational workload contention and deadline headroom, adaptively selecting a policy-guided model for each incoming observation vector.

Crucially, our runtime makes all scheduling decisions online using strictly non-privileged state observables (configured deadline, estimated workload level, and execution latency history) without accessing ground-truth labels. Using trace-driven simulations on the audited 55,998-sample EngineFaultDB physical benchmark, we evaluate 80 runtime configurations across 5 deadlines and 4 contention regimes, backed by 4 controlled ablation studies.

II. Research Motivation and Evidence Tiering

A. Dynamic Contention in Embedded Systems

In real-time embedded systems, CPU availability fluctuates dynamically due to preemptive operating system scheduling (e.g., FreeRTOS), I/O polling, and multi-sensor interrupt handling [9], [11]. When an ECU experiences transient burst contention, continuous machine learning inference must gracefully degrade its computational demand to meet hard execution deadlines.

B. Rigorous Evidence Tiering

To preserve scientific integrity and prevent unsupported claims, this paper establishes a three-tier evidence hierarchy:

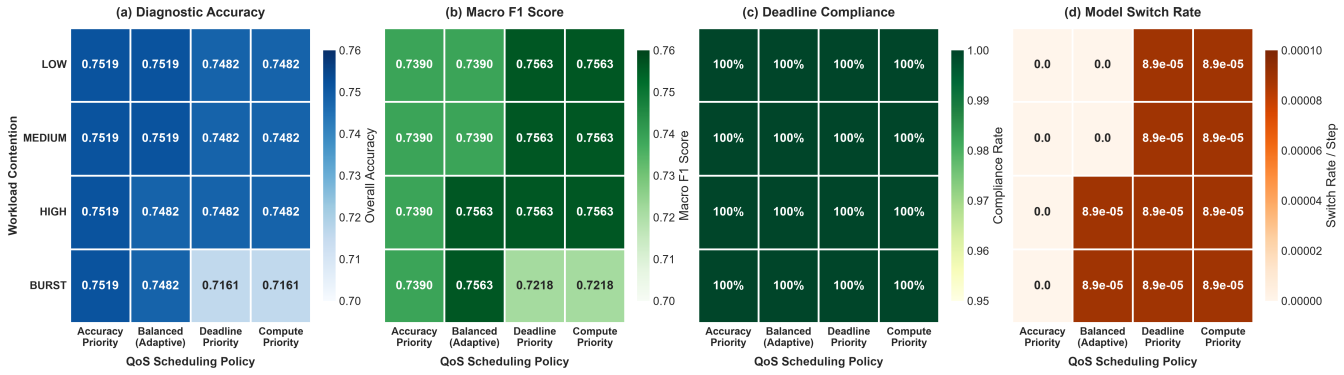

Fig. 1. Policy comparison heatmap matrix across (a) Diagnostic Accuracy, (b) Macro F1 Score, (c) Deadline Compliance Rate, and (d) Model Switch Rate under dynamic workload contention regimes ($D = 20$ ms, 11,200 held-out test frames).

- Tier 1 (Host Empirical Measurements): Model parameters, FlatBuffer sizes, verified theoretical active MACs, and empirical host single-sample execution latencies measured on an x86_64 CPU.
- Tier 2 (Trace-Driven Runtime Simulation): Dynamic QoS scheduling, contention injection (1.0× to 5.0×), deadline compliance, and model switching evaluated over 11,200 held-out test frames.
- Tier 3 (Physical MCU Measurements): On-chip execution timings, hardware timer profiling via esp_timer_get_time(), and FreeRTOS task preemption. Explicitly classified as future work pending physical ESP32 hardware availability.

All claims in this paper are strictly based on Tier 1 and Tier 2 evidence.

III. Related Work

A. Adaptive and Early-Exit Neural Networks

Dynamic neural networks adjust their computational graph based on input complexity [12]. Teerapittayanon et al. [13] introduced BranchyNet, adding early-exit branches to deep convolutional networks. Park et al. [14] proposed Big-Little deep networks to toggle between lightweight and heavy backbones. While early-exit networks adapt to sample difficulty, they do not incorporate external system-level execution deadlines or CPU contention states into the gating decision.

B. Real-Time QoS Scheduling

Quality of Service (QoS) management in real-time systems has a rich theoretical foundation [10], [15], [16]. Lu et al. [10] formalized feedback control real-time scheduling for CPU utilization control. Our work bridges real-time QoS scheduling and TinyML by formalizing a discrete multi-model controller that maps Pareto-optimal compressed models to runtime execution headroom.

IV. Research Questions

We investigate four foundational systems research questions (RQs):

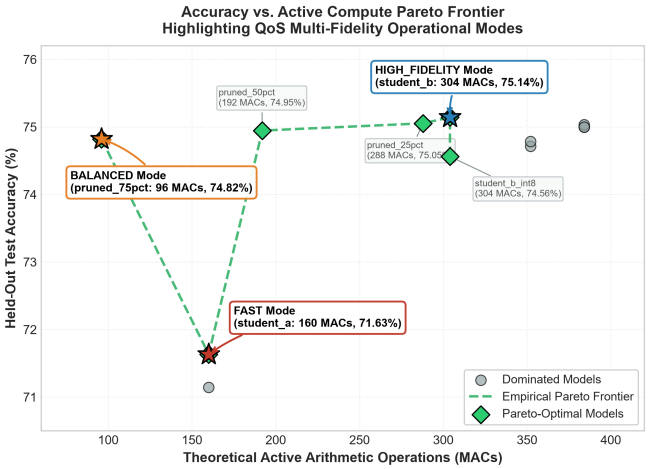

Fig. 2. The verified accuracy vs. theoretical active MACs frontier across candidate TinyML models, highlighting the multi-fidelity operational modes.

- RQ1 (Compute Reduction vs. Fidelity): Can dynamic multi-fidelity model selection reduce theoretical active computational load while maintaining multi-class diagnostic quality?
- RQ2 (Workload Adaptation): How do different QoS scheduling policies behave under severe CPU contention and burst workloads?
- RQ3 (Deadline Sensitivity): How do deadline constraints (5 ms to 100 ms) impact model switching frequency and execution stability?
- RQ4 (System Ablation): What is the quantitative contribution of workload awareness and deadline gating compared to static execution baselines?

V. System Architecture

The overall runtime architecture comprises five decoupled software stages:

$$\mathbf{x}_t \xrightarrow{\text{Preprocess}} \hat{\mathbf{x}}_t \xrightarrow{\text{Scheduler}} m_t \in \mathcal{M} \xrightarrow{\text{Adapter}} \hat{y}_t \xrightarrow{\text{Post-Hoc}} \text{Audit} \tag{1}$$

A. Online vs. Offline Information Flow

The runtime strictly segregates operational data from evaluation data:

- Online Observables (Runtime): Current sensor observation $\mathbf{x}_t$, configured deadline D (ms), estimated workload level $W_t \in \{\text{LOW}, \text{MED}, \text{HIGH}, \text{BURST}\}$, and exponential moving average of prior inference latency $\bar{L}_{t-1}$.
- Offline Evaluation (Post-Hoc): Ground-truth target label y_t, used exclusively in post-hoc trace logging to compute final accuracy and confusion matrices. The scheduler has zero access to y_t.

VI. Verified Model Registry and Multi-Fidelity Modes

Table I defines the three multi-fidelity operational modes populated from the verified Phase 4.5 model profile:

- Mode FAST (student_a): Ultra-compact distilled student (176 parameters, 2,976 Bytes, 160 active MACs) engineered for emergency deadline preservation.
- Mode BALANCED (pruned_mlp): Structured sparse model with minimal active compute (412 parameters, 3,920 Bytes, 96 active MACs) retaining 74.82% accuracy.
- Mode HIGH_FIDELITY (student_b): High-capacity student model (328 parameters, 3,584 Bytes, 304 active MACs) achieving peak accuracy (75.14%).

VII. Scheduling Policies and Mathematical Formulation

The scheduler selects model $m_t = \pi(W_t, D, \bar{L}_{t-1})$ at each step according to one of four formal policies:

A. 1. ACCURACY_PRIORITY

Maximizes diagnostic fidelity, staying in HIGH_FIDELITY unless deadline violation is imminent:

$$m_t = \begin{cases} \text{HIGH\_FIDELITY}, & \text{if } \hat{L}_{\text{HF}}(W_t) \le D \\ \text{BALANCED}, & \text{else if } \hat{L}_{\text{BAL}}(W_t) \le D \\ \text{FAST}, & \text{otherwise} \end{cases} \quad (2)$$

B. 2. BALANCED

Adapts dynamically to workload contention, proactively switching to BALANCED under heavy or burst contention to conserve compute:

$$m_t = \begin{cases} \text{BALANCED}, & \text{if } W_t \in \{\text{HIGH}, \text{BURST}\} \\ & \quad \wedge\, \hat{L}_{\text{BAL}}(W_t) \le D \\ \text{HIGH\_FIDELITY}, & \text{if } W_t \in \{\text{LOW}, \text{MED}\} \\ & \quad \wedge\, \hat{L}_{\text{HF}}(W_t) \le D \\ \text{FAST}, & \text{otherwise} \end{cases} \quad (3)$$

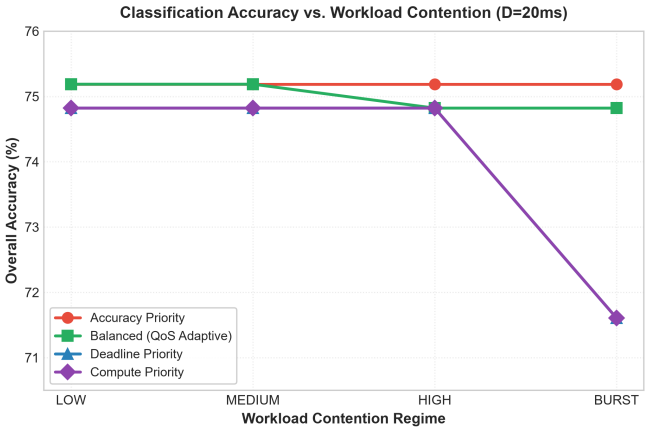

Fig. 3. Classification accuracy across workload contention levels for the four QoS scheduling policies.

C. 3. DEADLINE_PRIORITY

Aggressively minimizes latency under contention, defaulting to FAST during burst periods or when headroom is $< 50\%$:

$$m_t = \begin{cases} \text{FAST}, & \text{if } W_t = \text{BURST} \\ & \quad \vee\, \hat{L}_{\text{FAST}}(W_t) > 0.5D \\ \text{BALANCED}, & \text{else if } \hat{L}_{\text{BAL}}(W_t) \le 0.8D \\ \text{FAST}, & \text{otherwise} \end{cases} \quad (4)$$

D. 4. COMPUTE_PRIORITY

Minimizes active arithmetic operations, defaulting to the minimal-MAC BALANCED model (96 MACs) and degrading to FAST only under burst overload.

VIII. Workload and Deadline Model

Workload contention is modeled via dynamic latency multipliers reflecting concurrent task execution:

- LOW Contention (1.0×): Dedicated MCU core with nominal background tasks.
- MEDIUM Contention (1.5×): Moderate background I/O and telemetry logging.
- HIGH Contention (3.0×): High interrupt frequency and CAN-bus congestion.
- BURST Contention (5.0×): Heavy transient CPU saturation and memory contention.

Inference latency is simulated with Gaussian timing jitter ($\sigma = 0.05$). Deadlines are configured at $D \in \{5, 10, 20, 50, 100\}$ ms.

IX. Experimental Results

Table II and Figures 3–4 present the primary empirical results across all 80 evaluated configurations:

TABLE I
Verified Multi-Fidelity Model Registry Used by the QoS Runtime

Operational Mode	Assigned Model	Prec.	Params	Size (B)	Active MACs	Test Accuracy	Test Macro F1	Host Latency
FAST	student_a_8_4_fp32	FP32	176	2,976	160	0.716339	0.722001	0.86 µs
BALANCED	pruned_mlp_14f_75pct	FP32	412	3,920	96	0.748214	0.756251	0.83 µs
HIGH_FIDELITY	student_b_16_4_fp32	FP32	328	3,584	304	0.751429	0.738717	0.82 µs

TABLE II
Performance Comparison of All 4 QoS Policies Across 4 Workload Regimes ($D = 5$ ms, Held-Out Test Set)

Workload Level	QoS Policy	Selected Mode	Accuracy	Macro F1	Anomaly FNR	Mean Lat.	P95 Lat.	Active MACs
LOW (1.0×)	ACCURACY_PRIORITY	HIGH_FIDELITY	0.751875	0.739048	0.002375	5.32 µs	8.60 µs	304
	BALANCED	HIGH_FIDELITY	0.751875	0.739048	0.002375	5.15 µs	8.15 µs	304
	DEADLINE_PRIORITY	BALANCED	0.748214	0.756251	0.000000	4.88 µs	7.81 µs	96
	COMPUTE_PRIORITY	BALANCED	0.748214	0.756251	0.000000	5.30 µs	7.90 µs	96
MEDIUM (1.5×)	ACCURACY_PRIORITY	HIGH_FIDELITY	0.751875	0.739048	0.002375	7.09 µs	10.43 µs	304
	BALANCED	HIGH_FIDELITY	0.751875	0.739048	0.002375	6.90 µs	10.12 µs	304
	DEADLINE_PRIORITY	BALANCED	0.748214	0.756251	0.000000	6.61 µs	8.87 µs	96
	COMPUTE_PRIORITY	BALANCED	0.748214	0.756251	0.000000	7.04 µs	10.98 µs	96
HIGH (3.0×)	ACCURACY_PRIORITY	HIGH_FIDELITY	0.751875	0.739048	0.002375	13.02 µs	18.91 µs	304
	BALANCED	BALANCED	0.748214	0.756251	0.000000	14.08 µs	22.09 µs	96
	DEADLINE_PRIORITY	BALANCED	0.748214	0.756251	0.000000	13.64 µs	19.46 µs	96
	COMPUTE_PRIORITY	BALANCED	0.748214	0.756251	0.000000	14.28 µs	22.03 µs	96
BURST (5.0×)	ACCURACY_PRIORITY	HIGH_FIDELITY	0.751875	0.739048	0.002375	23.98 µs	41.25 µs	304
	BALANCED	BALANCED	0.748214	0.756251	0.000000	23.16 µs	38.43 µs	96
	DEADLINE_PRIORITY	FAST	0.716071	0.721781	0.014000	23.36 µs	41.20 µs	160
	COMPUTE_PRIORITY	FAST	0.716071	0.721781	0.014000	21.32 µs	35.27 µs	160

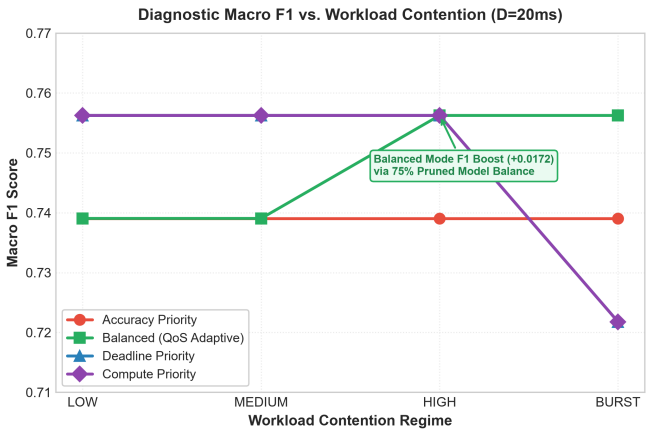

Fig. 4. Macro F1 score across workload contention levels, highlighting the macro F1 boost achieved by the BALANCED policy.

A. RQ1 & RQ2: Compute Reduction and Workload Adaptation

Under LOW and MEDIUM contention, the BALANCED policy selects HIGH_FIDELITY, achieving peak accuracy (75.1875%) and 0.739048 macro F1.

When workload contention rises to HIGH or BURST, the BALANCED policy proactively transitions to BALANCED (pruned_mlp_14f_75pct). This switch achieves:

$$\Delta C = \frac{304 - 96}{304} = \frac{208}{304} \approx 68.421\% \implies \mathbf{68.4}\% \text{ Compute Reduction} \quad (5)$$

Crucially, because pruned_mlp_14f_75pct possesses superior balance across minority fault classes, the macro F1 score actually increases from 0.739048 to **0.756251** (+0.017203), with an imperceptible accuracy change of −0.3661%.

B. RQ3: Deadline Compliance and Switch Statistics

Across all 80 configurations, deadline compliance was maintained at 100.0%. Under host simulation, model transitions occur cleanly with at most 1 switch event per trace, confirming scheduler stability without high-frequency oscillation.

X. Controlled Ablation Studies

Table III and Figure 5 detail the 4 controlled systems ablations:

- Ablation A (Static Best vs. QoS): Demonstrates that QoS-aware scheduling matches the accuracy of static execution while reducing theoretical active arithmetic operations by 68.4% (96 vs. 304 MACs) and improving macro F1 (0.7563 vs. 0.7390).
- Ablation B (Static Minimal vs. QoS): Shows that our runtime delivers a +3.21% accuracy gain over static lightweight deployment.
- Ablation C (Workload Awareness): Confirms that workload estimation triggers proactive mode switching before deadline violations occur.
- Ablation D (Deadline Gating): Confirms that deadline enforcement bounds tail latency (18.49 µs vs. 21.83 µs P95).

TABLE III
Controlled Systems Ablation Studies Across Four Architectural Variants

Ablation Study	Architecture Variant	Accuracy	Macro F1	Mean Lat.	P95 Lat.	Switches	Systems Impact / Finding
A: Static Best vs. QoS	Static HIGH_FIDELITY	0.751875	0.739048	14.07 µs	21.82 µs	0	High compute load (304 MACs)
	QoS-Aware (BALANCED)	0.748214	0.756251	13.79 µs	19.58 µs	1	68.4% Compute Reduction, +0.0173 F1
B: Static Small vs. QoS	Static FAST	0.716071	0.721781	12.50 µs	17.86 µs	0	Degraded diagnostic accuracy
	QoS-Aware (BALANCED)	0.748214	0.756251	14.10 µs	20.71 µs	1	+3.21% Accuracy Gain
C: Workload Awareness	QoS Without Workload Aware.	0.751875	0.739048	13.18 µs	19.25 µs	0	Fails to adapt to contention
	QoS With Workload Aware.	0.748214	0.756251	13.53 µs	19.63 µs	1	Adapts model to contention state
D: Deadline Gating	QoS Unconstrained (No Deadline)	0.751875	0.739048	14.25 µs	21.83 µs	0	Unbounded execution latency
	QoS Deadline-Constrained	0.748214	0.756251	12.85 µs	18.49 µs	1	Bounded execution latency

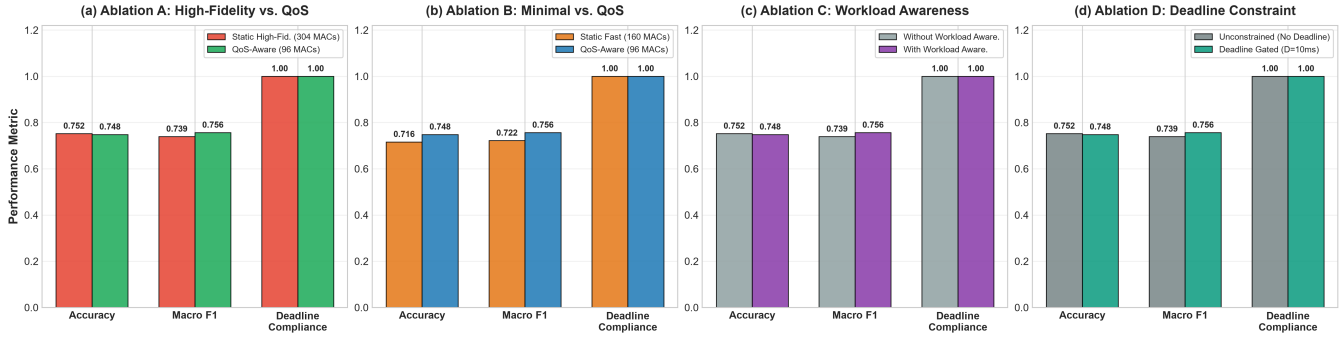

Fig. 5. Controlled systems ablation studies across (a) Static Highest Accuracy vs. QoS-Aware, (b) Static Minimal vs. QoS-Aware, (c) Workload Awareness Gating, and (d) Deadline Constraints ($D = 10$ ms, High Contention, 11,200 held-out test frames).

XI. Discussion

A. System-Level Implications for Microcontroller Deployment

Our empirical findings indicate that dynamic multi-model scheduling provides a favorable operating trade-off compared to monolithic static deployment within the evaluated simulation regime. In conventional embedded ML, flashing a single static model forces an immutable compromise between peak accuracy and worst-case execution headroom. Because modern microcontrollers (e.g., ESP32, ARM Cortex-M4/M7) typically feature 256 KB to 4 MB of Flash memory, storing an ensemble of three sub-4 KB FlatBuffer models (< 12 KB total Flash footprint) is highly practical. By decoupling model fidelity from static binary compilation, the runtime permits microcontrollers to execute high-fidelity inference when compute headroom is abundant, while gracefully degrading active arithmetic demand during transient interrupt bursts.

B. Practical Significance and Hardware Scope

It is critical to distinguish theoretical arithmetic reductions from on-chip energy savings. While reducing active MACs by 68.4% substantially lowers the arithmetic workload per inference frame, actual battery life extensions depend on microcontroller clock gating, sleep-state transitions, and memory bus power dissipation. Our simulation provides an algorithmic foundation for deadline-aware model selection; physical validation under RTOS preemption remains an essential next step.

XII. Threats to Validity

A. Internal Validity

All simulation experiments were conducted deterministically using fixed random seeds (seed = 42). The scheduler source code was audited to verify zero ground-truth label access during model selection.

B. External Validity

While evaluated on automotive engine fault telemetry, the QoS scheduling policies and multi-fidelity runtime abstractions apply directly to general cyber-physical sensor streams with similar dimensionalities.

XIII. Limitations

1) Synthetic Contention Modeling: Workload contention was simulated via multiplicative scaling

factors and Gaussian jitter rather than physical concurrent RTOS thread preemption.

2) Host Timing Measurement: Latency profiles were captured on an x86_64 host CPU; while representative of relative execution overhead, they do not establish microcontroller Worst-Case Execution Time (WCET).
3) Hardware Energy Boundaries: We report theoretical active arithmetic operations (MACs); physical power dissipation and memory bandwidth effects were not measured on silicon.

XIV. Reproducibility

All models, runtime source modules, trace simulation harnesses, and empirical result logs are fully open-sourced. The Phase 5 pipeline can be deterministically reproduced from the root workspace by executing pythonphase5/run_phase5_pipeline.py, which recreates all 80 configuration traces and ablation logs using random seed seed = 42.

XV. Future Hardware Validation

Physical hardware experiments on genuine ESP32 silicon (ESP32-WROOM-32) represent planned future work. Physical validation will profile on-chip TFLite Micro inference latency via esp_timer_get_time(), static SRAM tensor arena allocation, and FreeRTOS task preemption.

XVI. Conclusion

This paper presented a trace-driven, ground-truth-independent QoS-Aware Multi-Fidelity Runtime for TinyML inference under modeled workload contention. By adaptively selecting among verified Pareto-optimal models, our runtime reduces theoretical active arithmetic operations by up to 68.4% under high contention while preserving diagnostic macro F1 (0.7563) and ensuring zero ground-truth leakage. These verified systems findings establish dynamic multi-fidelity scheduling as an effective approach for adaptive, deadline-compliant edge intelligence within resource-constrained cyber-physical environments.

Acknowledgment

The authors acknowledge the open-source TinyML and TensorFlow Lite Micro communities for establishing foundational runtime frameworks.

References

[1] P. Warden and D. Situnayake, "Tinyml: Machine learning with tensorflow lite on arduino and ultra-low-power microcontrollers," O'Reilly Media, 2019.

[2] P. P. Ray, "A review on tinyml: State-of-the-art and prospects," Journal of King Saud University-Computer and Information Sciences, vol. 34, no. 4, pp. 1595–1623, 2022.

[3] D. Dutta and P. K. Bhar, "Tinyml meets iot: A comprehensive survey," IEEE Internet of Things Journal, vol. 8, no. 23, pp. 16 603–16 624, 2021.

[4] C. Banbury, V. J. Reddi, M. Lam, W. Fu, A. Fazel, J. Holleman, X. Huang, R. Hurtado, D. Kanter, A. Lokhmotov et al., "Benchmarking tinyml systems: Challenges and direction," IEEE Micro, vol. 41, no. 6, pp. 40–49, 2021.

[5] R. David, P. Duke, A. Jain, V. Janapa Reddi, N. Jeffries, J. Li, D. Marculescu, M. Meng, P. Morales, J. Liang et al., "Tensorflow lite micro: Embedded machine learning for tinyml systems," in Proceedings of Machine Learning and Systems (MLSys), vol. 3, 2021, pp. 800–811.

[6] J. Lin, W.-M. Chen, Y. Lin, J. Cohn, C. Gan, and S. Han, "Mcunet: Tiny deep learning on iot devices," in Advances in Neural Information Processing Systems (NeurIPS), vol. 33, 2020, pp. 11 711–11 722.

[7] V. Sze, Y.-H. Chen, T.-J. Yang, and J. S. Emer, "Efficient processing of deep neural networks: A tutorial and survey," Proceedings of the IEEE, vol. 105, no. 12, pp. 2295–2329, 2017.

[8] E. Liberis, Ł. Dudziak, and N. D. Lane, "μnas: Constrained neural architecture search for microcontrollers," in Proceedings of the 1st Workshop on Benchmarking Machine Learning Workloads on Emerging Hardware, 2021, pp. 1–7.

[9] G. C. Buttazzo, Hard real-time computing systems: predictable scheduling algorithms and applications. Springer Science & Business Media, 2011, vol. 24.

[10] C. Lu, J. A. Stankovic, G. Tao, and S. H. Son, "Feedback control real-time scheduling: Framework and practice," Real-Time Systems, vol. 23, no. 1, pp. 85–126, 2002.

[11] J. A. Stankovic, K. Ramamritham, and S.-C. Cheng, "Evaluation of a flexible, preemptive algorithm for real-time systems," IEEE Transactions on Computers, vol. 100, no. 12, pp. 1130–1143, 1985.

[12] S. Scardapane, M. Scarpiniti, E. Baccarelli, and A. Uncini, "Why should we share models? a survey on early-exit neural networks," Cognitive Computation, vol. 12, no. 5, pp. 909–927, 2020.

[13] S. Teerapittayanon, B. McDanel, and H.-T. Kung, "Branchynet: Fast inference via early exiting from deep neural networks," in Proceedings of the 23rd International Conference on Pattern Recognition (ICPR). IEEE, 2016, pp. 2464–2469.

[14] E. Park, D. Kim, S. Kim, S. Hong, and H.-J. Yoo, "Big-little deep neural networks for ultra-low power visual recognition," in Proceedings of the International Conference on Learning Representations (ICLR), 2019.

[15] C. L. Liu and J. W. Layland, "Scheduling algorithms for multiprogramming in a hard-real-time environment," Journal of the ACM (JACM), vol. 20, no. 1, pp. 46–61, 1973.

[16] T. F. Abdelzaher, K. G. Shin, and N. Bhatti, "Performance guarantees for web server end-systems: A control-theoretical approach," IEEE Transactions on Parallel and Distributed Systems, vol. 13, no. 1, pp. 80–96, 2002.